

WAMU — UGANDA SUPER APP

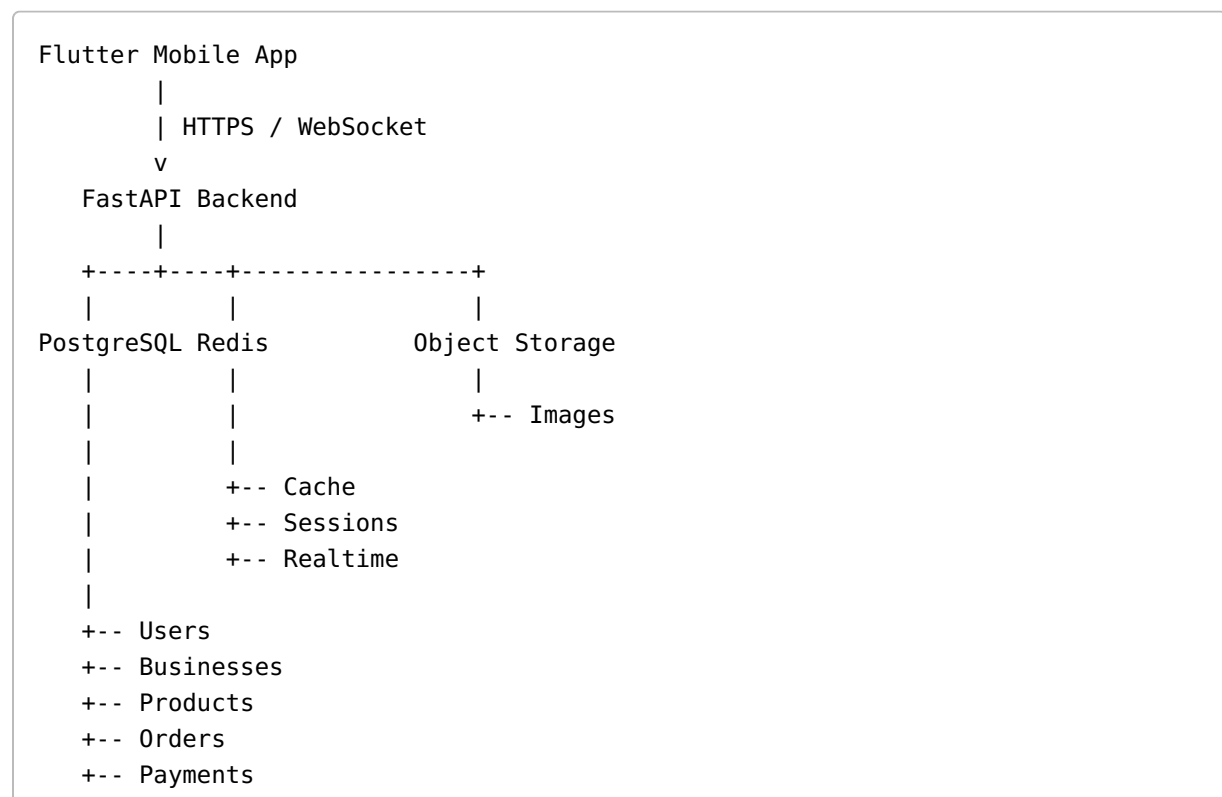
Part 2 — PostgreSQL Database, ER Diagram, FastAPI Models & API Schemas

1. Architecture

The backend will use:

- **Python**
- **FastAPI**
- **PostgreSQL**
- **SQLAlchemy 2.x**
- **Alembic**
- **Pydantic v2**
- **PostGIS** later for advanced location search
- **Redis** for caching/realtime
- **JWT** for authentication

Basic architecture:



- +-- Chat
- +-- Reviews
- +-- Notifications

2. Database Design Principles

Use these rules from the beginning:

1. Every primary key is a UUID.
2. Never expose sequential database IDs publicly.
3. Store money using `NUMERIC`, never floating-point.
4. Store timestamps in UTC.
5. Use foreign keys.
6. Use database constraints in addition to API validation.
7. Use indexes for frequently searched columns.
8. Use soft deletion where appropriate.
9. Never store passwords in plaintext.
10. Payment records must be immutable/auditable.
11. Never trust values coming from the mobile application.
12. Business verification and user permissions must be enforced server-side.

3. Main Tables

The initial database contains:

```
users
user_profiles
sessions

categories
businesses
business_members

products
product_images

locations

orders
order_items
```

payments

conversations

conversation_members

messages

reviews

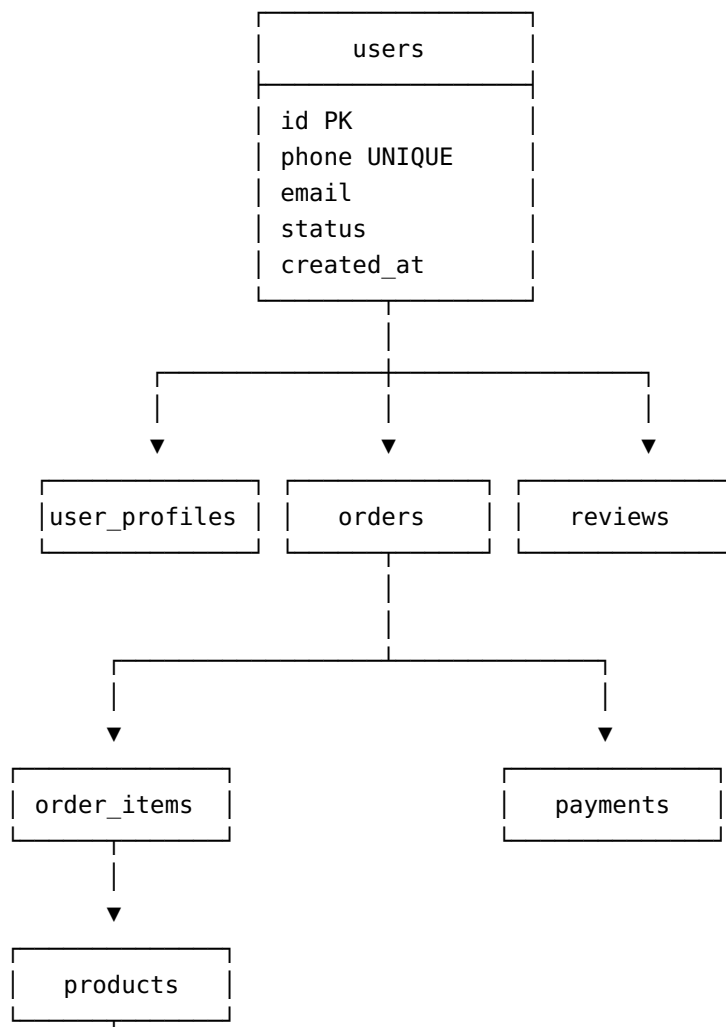
favorites

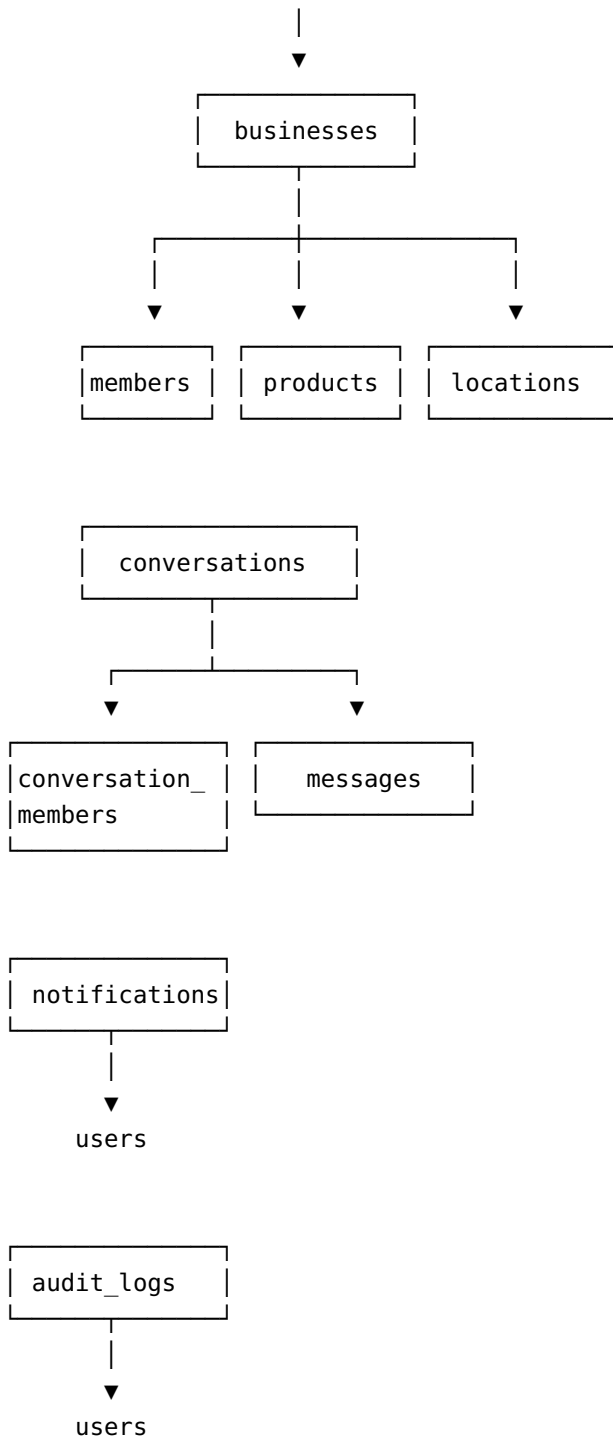
notifications

reports

audit_logs

4. ER Diagram





5. PostgreSQL SQL Schema

5.1 Extensions

```
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
```

If advanced geographic search is introduced:

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

6. Users

```
CREATE TYPE user_status AS ENUM (  
    'ACTIVE',  
    'SUSPENDED',  
    'DELETED'  
);  
  
CREATE TABLE users (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    phone VARCHAR(20) NOT NULL UNIQUE,  
  
    email VARCHAR(255) UNIQUE,  
  
    status user_status NOT NULL DEFAULT 'ACTIVE',  
  
    phone_verified BOOLEAN NOT NULL DEFAULT FALSE,  
  
    email_verified BOOLEAN NOT NULL DEFAULT FALSE,  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_users_phone  
ON users(phone);
```

```
CREATE INDEX idx_users_email
ON users(email);
```

7. User Profiles

```
CREATE TABLE user_profiles (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  user_id UUID NOT NULL UNIQUE
    REFERENCES users(id) ON DELETE CASCADE,

  first_name VARCHAR(100),

  last_name VARCHAR(100),

  username VARCHAR(50) UNIQUE,

  avatar_url TEXT,

  bio TEXT,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_user_profiles_username
ON user_profiles(username);
```

Relationship:

```
users 1 ————— 1 user_profiles
```

8. Sessions

```
CREATE TABLE sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  user_id UUID NOT NULL
```

```

        REFERENCES users(id) ON DELETE CASCADE,

refresh_token_hash TEXT NOT NULL,

device_name VARCHAR(255),

device_id VARCHAR(255),

ip_address INET,

expires_at TIMESTAMPTZ NOT NULL,

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

revoked_at TIMESTAMPTZ
);

CREATE INDEX idx_sessions_user_id
ON sessions(user_id);

CREATE INDEX idx_sessions_expires_at
ON sessions(expires_at);

```

Do not store raw refresh tokens.

9. Categories

Categories allow Wamu to organize businesses and products.

```

CREATE TABLE categories (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  name VARCHAR(100) NOT NULL UNIQUE,

  slug VARCHAR(120) NOT NULL UNIQUE,

  description TEXT,

  icon_url TEXT,

  parent_id UUID REFERENCES categories(id),

  is_active BOOLEAN NOT NULL DEFAULT TRUE,

```

```

        created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
    );

CREATE INDEX idx_categories_parent
ON categories(parent_id);

```

Example:

```

Food
├── Restaurants
├── Fast Food
├── Cakes
└── Groceries

Fashion
├── Men's Fashion
├── Women's Fashion
└── Shoes

Services
├── Plumbing
├── Electrical
├── Cleaning
└── Construction

```

10. Businesses

```

CREATE TYPE business_status AS ENUM (
    'ACTIVE',
    'INACTIVE',
    'SUSPENDED',
    'DELETED'
);

CREATE TYPE verification_status AS ENUM (
    'UNVERIFIED',
    'PENDING',
    'VERIFIED',
    'REJECTED'
);

CREATE TABLE businesses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

owner_id UUID NOT NULL
    REFERENCES users(id),

category_id UUID
    REFERENCES categories(id),

name VARCHAR(200) NOT NULL,

slug VARCHAR(220) NOT NULL UNIQUE,

description TEXT,

phone VARCHAR(20),

email VARCHAR(255),

logo_url TEXT,

cover_image_url TEXT,

status business_status NOT NULL DEFAULT 'ACTIVE',

verification_status verification_status
    NOT NULL DEFAULT 'UNVERIFIED',

rating NUMERIC(3,2) NOT NULL DEFAULT 0,

review_count INTEGER NOT NULL DEFAULT 0,

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_businesses_owner
ON businesses(owner_id);

CREATE INDEX idx_businesses_category
ON businesses(category_id);

CREATE INDEX idx_businesses_status
ON businesses(status);

CREATE INDEX idx_businesses_verification
ON businesses(verification_status);

```

11. Business Members

A business can have multiple employees/admins.

```
CREATE TYPE business_role AS ENUM (  
    'OWNER',  
    'ADMIN',  
    'STAFF'  
);  
  
CREATE TABLE business_members (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    business_id UUID NOT NULL  
        REFERENCES businesses(id) ON DELETE CASCADE,  
  
    user_id UUID NOT NULL  
        REFERENCES users(id) ON DELETE CASCADE,  
  
    role business_role NOT NULL DEFAULT 'STAFF',  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
    UNIQUE(business_id, user_id)  
);  
  
CREATE INDEX idx_business_members_business  
ON business_members(business_id);  
  
CREATE INDEX idx_business_members_user  
ON business_members(user_id);
```

Relationship:

users 1 ————— * business_members * ————— 1 businesses

12. Locations

```
CREATE TABLE locations (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

business_id UUID UNIQUE
REFERENCES businesses(id) ON DELETE CASCADE,

address_line TEXT,

area VARCHAR(150),

city VARCHAR(100),

district VARCHAR(100),

country VARCHAR(100) NOT NULL DEFAULT 'Uganda',

latitude NUMERIC(10,7),

longitude NUMERIC(10,7),

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_locations_city
ON locations(city);

CREATE INDEX idx_locations_district
ON locations(district);

CREATE INDEX idx_locations_coordinates
ON locations(latitude, longitude);

```

Later, replace the latitude/longitude pair with PostGIS:

```
location GEOGRAPHY(POINT, 4326)
```

This will allow:

```

businesses within 2 km
businesses within 5 km
nearest restaurant
nearest pharmacy
nearest mechanic

```

13. Products

```
CREATE TYPE product_status AS ENUM (  
    'ACTIVE',  
    'INACTIVE',  
    'OUT_OF_STOCK',  
    'DELETED'  
);  
  
CREATE TABLE products (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    business_id UUID NOT NULL  
        REFERENCES businesses(id) ON DELETE CASCADE,  
  
    name VARCHAR(200) NOT NULL,  
  
    slug VARCHAR(220) NOT NULL,  
  
    description TEXT,  
  
    price NUMERIC(14,2) NOT NULL,  
  
    currency CHAR(3) NOT NULL DEFAULT 'UGX',  
  
    stock_quantity INTEGER,  
  
    status product_status NOT NULL DEFAULT 'ACTIVE',  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
    UNIQUE(business_id, slug),  
  
    CHECK(price >= 0),  
  
    CHECK(stock_quantity IS NULL OR stock_quantity >= 0)  
);  
  
CREATE INDEX idx_products_business  
ON products(business_id);  
  
CREATE INDEX idx_products_status  
ON products(status);
```

```
CREATE INDEX idx_products_name
ON products(name);
```

Example:

Business: FynCakes

Products:

Chocolate Cake
price = 85000
currency = UGX

Birthday Cake
price = 120000
currency = UGX

14. Product Images

```
CREATE TABLE product_images (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  product_id UUID NOT NULL
    REFERENCES products(id) ON DELETE CASCADE,

  image_url TEXT NOT NULL,

  position INTEGER NOT NULL DEFAULT 0,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_product_images_product
ON product_images(product_id);
```

Images should live in object storage, not PostgreSQL.

15. Orders

```
CREATE TYPE order_status AS ENUM (  
    'PENDING',  
    'CONFIRMED',  
    'PROCESSING',  
    'READY',  
    'OUT_FOR_DELIVERY',  
    'DELIVERED',  
    'CANCELLED',  
    'REFUNDED'  
);  
  
CREATE TABLE orders (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    customer_id UUID NOT NULL  
        REFERENCES users(id),  
  
    business_id UUID NOT NULL  
        REFERENCES businesses(id),  
  
    status order_status NOT NULL DEFAULT 'PENDING',  
  
    subtotal NUMERIC(14,2) NOT NULL,  
  
    delivery_fee NUMERIC(14,2) NOT NULL DEFAULT 0,  
  
    total NUMERIC(14,2) NOT NULL,  
  
    currency CHAR(3) NOT NULL DEFAULT 'UGX',  
  
    delivery_address TEXT,  
  
    customer_note TEXT,  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
    CHECK(subtotal >= 0),  
  
    CHECK(delivery_fee >= 0),  
  
    CHECK(total >= 0)  
);
```

```
CREATE INDEX idx_orders_customer
ON orders(customer_id);

CREATE INDEX idx_orders_business
ON orders(business_id);

CREATE INDEX idx_orders_status
ON orders(status);

CREATE INDEX idx_orders_created_at
ON orders(created_at);
```

16. Order Items

```
CREATE TABLE order_items (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  order_id UUID NOT NULL
    REFERENCES orders(id) ON DELETE CASCADE,

  product_id UUID NOT NULL
    REFERENCES products(id),

  quantity INTEGER NOT NULL,

  unit_price NUMERIC(14,2) NOT NULL,

  subtotal NUMERIC(14,2) NOT NULL,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CHECK(quantity > 0),

  CHECK(unit_price >= 0),

  CHECK(subtotal >= 0)
);

CREATE INDEX idx_order_items_order
ON order_items(order_id);
```

```
CREATE INDEX idx_order_items_product
ON order_items(product_id);
```

Important:

`unit_price` must be copied into the order item.

Why?

Because if a product costs:

Today: UGX 50,000
Tomorrow: UGX 60,000

old orders must still show:

UGX 50,000

17. Payments

```
CREATE TYPE payment_status AS ENUM (
    'PENDING',
    'PROCESSING',
    'SUCCESS',
    'FAILED',
    'CANCELLED',
    'REFUNDED'
);

CREATE TYPE payment_method AS ENUM (
    'MTN_MOMO',
    'AIRTEL_MONEY',
    'CARD'
);

CREATE TABLE payments (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    order_id UUID NOT NULL
        REFERENCES orders(id),
```

```

user_id UUID NOT NULL
      REFERENCES users(id),

provider VARCHAR(50) NOT NULL,

method payment_method NOT NULL,

provider_reference VARCHAR(255),

amount NUMERIC(14,2) NOT NULL,

currency CHAR(3) NOT NULL DEFAULT 'UGX',

status payment_status NOT NULL DEFAULT 'PENDING',

raw_response JSONB,

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

UNIQUE(provider, provider_reference),

CHECK(amount > 0)
);

CREATE INDEX idx_payments_order
ON payments(order_id);

CREATE INDEX idx_payments_user
ON payments(user_id);

CREATE INDEX idx_payments_status
ON payments(status);

```

Payment webhook processing must be **idempotent**.

Never create duplicate payments because a provider sends the same webhook twice.

18. Conversations

```

CREATE TABLE conversations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

    business_id UUID
        REFERENCES businesses(id) ON DELETE SET NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

```

A conversation may be:

```

Customer <-> Business
Customer <-> Customer

```

19. Conversation Members

```

CREATE TABLE conversation_members (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    conversation_id UUID NOT NULL
        REFERENCES conversations(id) ON DELETE CASCADE,

    user_id UUID NOT NULL
        REFERENCES users(id) ON DELETE CASCADE,

    joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

    UNIQUE(conversation_id, user_id)
);

CREATE INDEX idx_conversation_members_user
ON conversation_members(user_id);

CREATE INDEX idx_conversation_members_conversation
ON conversation_members(conversation_id);

```

20. Messages

```

CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

conversation_id UUID NOT NULL
    REFERENCES conversations(id) ON DELETE CASCADE,

sender_id UUID NOT NULL
    REFERENCES users(id),

message TEXT,

attachment_url TEXT,

is_read BOOLEAN NOT NULL DEFAULT FALSE,

created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_messages_conversation
ON messages(conversation_id, created_at);

CREATE INDEX idx_messages_sender
ON messages(sender_id);

```

For large-scale chat, `is_read` can later become a separate read-receipts table.

21. Reviews

```

CREATE TABLE reviews (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id UUID NOT NULL
        REFERENCES users(id),

    business_id UUID NOT NULL
        REFERENCES businesses(id) ON DELETE CASCADE,

    order_id UUID
        REFERENCES orders(id),

    rating INTEGER NOT NULL,

    comment TEXT,

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

```

```
    CHECK(rating BETWEEN 1 AND 5),

    UNIQUE(user_id, order_id)
);

CREATE INDEX idx_reviews_business
ON reviews(business_id);

CREATE INDEX idx_reviews_user
ON reviews(user_id);
```

22. Favorites

```
CREATE TABLE favorites (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

  user_id UUID NOT NULL
    REFERENCES users(id) ON DELETE CASCADE,

  business_id UUID
    REFERENCES businesses(id) ON DELETE CASCADE,

  product_id UUID
    REFERENCES products(id) ON DELETE CASCADE,

  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),

  CHECK (
    business_id IS NOT NULL
    OR product_id IS NOT NULL
  )
);
```

Later, separate this into:

```
favorite_businesses
favorite_products
```

if query performance requires it.

23. Notifications

```
CREATE TYPE notification_type AS ENUM (  
    'ORDER',  
    'PAYMENT',  
    'MESSAGE',  
    'PROMOTION',  
    'SYSTEM'  
);  
  
CREATE TABLE notifications (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    user_id UUID NOT NULL  
        REFERENCES users(id) ON DELETE CASCADE,  
  
    type notification_type NOT NULL,  
  
    title VARCHAR(255) NOT NULL,  
  
    body TEXT NOT NULL,  
  
    data JSONB,  
  
    is_read BOOLEAN NOT NULL DEFAULT FALSE,  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE INDEX idx_notifications_user  
ON notifications(user_id, created_at);  
  
CREATE INDEX idx_notifications_unread  
ON notifications(user_id, is_read);
```

24. Reports

Users need to report:

```
Business  
Product  
User
```

Message
Review

```
CREATE TYPE report_status AS ENUM (  
    'PENDING',  
    'REVIEWING',  
    'RESOLVED',  
    'DISMISSED'  
);  
  
CREATE TABLE reports (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
    reporter_id UUID NOT NULL  
        REFERENCES users(id),  
  
    target_type VARCHAR(50) NOT NULL,  
  
    target_id UUID NOT NULL,  
  
    reason VARCHAR(255) NOT NULL,  
  
    description TEXT,  
  
    status report_status NOT NULL DEFAULT 'PENDING',  
  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
    resolved_at TIMESTAMPTZ  
);  
  
CREATE INDEX idx_reports_status  
ON reports(status);  
  
CREATE INDEX idx_reports_reporter  
ON reports(reporter_id);
```

25. Audit Logs

This is especially important for payments, administration and security.

```
CREATE TABLE audit_logs (  
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

    user_id UUID
        REFERENCES users(id) ON DELETE SET NULL,

    action VARCHAR(100) NOT NULL,

    entity_type VARCHAR(100),

    entity_id UUID,

    old_values JSONB,

    new_values JSONB,

    ip_address INET,

    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_audit_logs_user
ON audit_logs(user_id);

CREATE INDEX idx_audit_logs_entity
ON audit_logs(entity_type, entity_id);

CREATE INDEX idx_audit_logs_created
ON audit_logs(created_at);

```

26. Complete Relationship Map

```

USER
|
+---- USER_PROFILE
|
+---- SESSIONS
|
+---- BUSINESS_MEMBERS
|           |
|           +---- BUSINESS
|                   |
|                   +---- CATEGORY
|                   |
|                   +---- LOCATION
|                   |

```

```

|                                     +----- PRODUCTS
|                                     |
|                                     +----- PRODUCT_IMAGES
|
+----- ORDERS
|   |
|   +----- ORDER_ITEMS
|   |   |
|   |   +----- PRODUCT
|   |
|   +----- PAYMENT
|
+----- CONVERSATION_MEMBERS
|   |
|   +----- CONVERSATION
|       |
|       +----- MESSAGES
|
+----- REVIEWS
|
+----- FAVORITES
|
+----- NOTIFICATIONS
|
+----- REPORTS
|
+----- AUDIT_LOGS

```

27. FastAPI Project Structure

```

wamu-backend/
|
├── app/
|   ├── main.py
|   ├── core/
|   │   ├── config.py
|   │   ├── database.py
|   │   ├── security.py
|   │   └── logging.py
|   ├── models/
|   │   ├── user.py
|   │   └── business.py

```

```

├── category.py
├── product.py
├── order.py
├── payment.py
├── chat.py
├── review.py
└── notification.py

schemas/
├── auth.py
├── user.py
├── business.py
├── product.py
├── order.py
├── payment.py
├── chat.py
├── review.py
└── notification.py

api/
└── v1/
    ├── auth.py
    ├── users.py
    ├── businesses.py
    ├── categories.py
    ├── products.py
    ├── search.py
    ├── orders.py
    ├── payments.py
    ├── chat.py
    ├── reviews.py
    └── notifications.py

services/
├── auth_service.py
├── business_service.py
├── order_service.py
├── payment_service.py
├── chat_service.py
└── notification_service.py

repositories/
├── user_repository.py
├── business_repository.py
├── product_repository.py
└── order_repository.py

alembic/

```

```
|
|— tests/
|
|— Dockerfile
|— docker-compose.yml
|— pyproject.toml
|— .env
```

28. SQLAlchemy Base

Use SQLAlchemy 2.x style.

```
from datetime import datetime
from uuid import UUID, uuid4

from sqlalchemy import DateTime, func
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column

class Base(DeclarativeBase):
    pass

class UUIDMixin:
    id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        primary_key=True,
        default=uuid4,
    )

class TimestampMixin:
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        nullable=False,
    )

    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True),
        server_default=func.now(),
        onupdate=func.now(),
```

```
        nullable=False,  
    )
```

29. User Model

```
from sqlalchemy import Boolean, String  
from sqlalchemy.orm import Mapped, mapped_column, relationship  
  
class User(UUIDMixin, TimestampMixin, Base):  
    __tablename__ = "users"  
  
    phone: Mapped[str] = mapped_column(  
        String(20),  
        unique=True,  
        nullable=False,  
        index=True,  
    )  
  
    email: Mapped[str | None] = mapped_column(  
        String(255),  
        unique=True,  
        nullable=True,  
    )  
  
    status: Mapped[str] = mapped_column(  
        String(20),  
        default="ACTIVE",  
        nullable=False,  
    )  
  
    phone_verified: Mapped[bool] = mapped_column(  
        Boolean,  
        default=False,  
        nullable=False,  
    )  
  
    email_verified: Mapped[bool] = mapped_column(  
        Boolean,  
        default=False,  
        nullable=False,  
    )  
  
    profile = relationship(  

```

```
        "UserProfile",
        back_populates="user",
        uselist=False,
        cascade="all, delete-orphan",
    )
```

30. User Profile Model

```
class UserProfile(UUIDMixin, TimestampMixin, Base):
    __tablename__ = "user_profiles"

    user_id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("users.id"),
        unique=True,
        nullable=False,
    )

    first_name: Mapped[str | None] = mapped_column(
        String(100)
    )

    last_name: Mapped[str | None] = mapped_column(
        String(100)
    )

    username: Mapped[str | None] = mapped_column(
        String(50),
        unique=True,
    )

    avatar_url: Mapped[str | None] = mapped_column(
        Text
    )

    bio: Mapped[str | None] = mapped_column(
        Text
    )

    user = relationship(
        "User",
        back_populates="profile",
    )
```

31. Category Model

```
class Category(UUIDMixin, Base):
    __tablename__ = "categories"

    name: Mapped[str] = mapped_column(
        String(100),
        unique=True,
        nullable=False,
    )

    slug: Mapped[str] = mapped_column(
        String(120),
        unique=True,
        nullable=False,
    )

    description: Mapped[str | None] = mapped_column(
        Text
    )

    icon_url: Mapped[str | None] = mapped_column(
        Text
    )

    parent_id: Mapped[UUID | None] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("categories.id"),
    )

    is_active: Mapped[bool] = mapped_column(
        Boolean,
        default=True,
        nullable=False,
    )

    parent = relationship(
        "Category",
        remote_side="Category.id",
        back_populates="children",
    )

    children = relationship(
        "Category",
```

```
        back_populates="parent",
    )
```

32. Business Model

```
class Business(UUIDMixin, TimestampMixin, Base):
    __tablename__ = "businesses"

    owner_id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("users.id"),
        nullable=False,
    )

    category_id: Mapped[UUID | None] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("categories.id"),
    )

    name: Mapped[str] = mapped_column(
        String(200),
        nullable=False,
    )

    slug: Mapped[str] = mapped_column(
        String(220),
        unique=True,
        nullable=False,
    )

    description: Mapped[str | None] = mapped_column(
        Text
    )

    phone: Mapped[str | None] = mapped_column(
        String(20)
    )

    email: Mapped[str | None] = mapped_column(
        String(255)
    )

    logo_url: Mapped[str | None] = mapped_column(
        Text
    )
```

```

)

cover_image_url: Mapped[str | None] = mapped_column(
    Text
)

status: Mapped[str] = mapped_column(
    String(20),
    default="ACTIVE",
    nullable=False,
)

verification_status: Mapped[str] = mapped_column(
    String(20),
    default="UNVERIFIED",
    nullable=False,
)

rating: Mapped[Decimal] = mapped_column(
    Numeric(3, 2),
    default=0,
    nullable=False,
)

review_count: Mapped[int] = mapped_column(
    Integer,
    default=0,
    nullable=False,
)

owner = relationship("User")

category = relationship("Category")

products = relationship(
    "Product",
    back_populates="business",
    cascade="all, delete-orphan",
)

location = relationship(
    "Location",
    back_populates="business",
    uselist=False,
    cascade="all, delete-orphan",
)

```

33. Product Model

```
class Product(UUIDMixin, TimestampMixin, Base):
    __tablename__ = "products"

    business_id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("businesses.id"),
        nullable=False,
    )

    name: Mapped[str] = mapped_column(
        String(200),
        nullable=False,
    )

    slug: Mapped[str] = mapped_column(
        String(220),
        nullable=False,
    )

    description: Mapped[str | None] = mapped_column(
        Text
    )

    price: Mapped[Decimal] = mapped_column(
        Numeric(14, 2),
        nullable=False,
    )

    currency: Mapped[str] = mapped_column(
        String(3),
        default="UGX",
        nullable=False,
    )

    stock_quantity: Mapped[int | None] = mapped_column(
        Integer
    )

    status: Mapped[str] = mapped_column(
        String(20),
        default="ACTIVE",
        nullable=False,
    )
```

```
business = relationship(
    "Business",
    back_populates="products",
)
```

34. Order Model

```
class Order(UUIDMixin, TimestampMixin, Base):
    __tablename__ = "orders"

    customer_id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("users.id"),
        nullable=False,
    )

    business_id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("businesses.id"),
        nullable=False,
    )

    status: Mapped[str] = mapped_column(
        String(30),
        default="PENDING",
        nullable=False,
    )

    subtotal: Mapped[Decimal] = mapped_column(
        Numeric(14, 2),
        nullable=False,
    )

    delivery_fee: Mapped[Decimal] = mapped_column(
        Numeric(14, 2),
        default=0,
        nullable=False,
    )

    total: Mapped[Decimal] = mapped_column(
        Numeric(14, 2),
        nullable=False,
    )
```

```

currency: Mapped[str] = mapped_column(
    String(3),
    default="UGX",
    nullable=False,
)

delivery_address: Mapped[str | None] = mapped_column(
    Text
)

customer_note: Mapped[str | None] = mapped_column(
    Text
)

customer = relationship("User")

business = relationship("Business")

items = relationship(
    "OrderItem",
    cascade="all, delete-orphan",
)

```

35. Order Item Model

```

class OrderItem(UUIDMixin, Base):
    __tablename__ = "order_items"

    order_id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("orders.id"),
        nullable=False,
    )

    product_id: Mapped[UUID] = mapped_column(
        UUID(as_uuid=True),
        ForeignKey("products.id"),
        nullable=False,
    )

    quantity: Mapped[int] = mapped_column(
        Integer,
        nullable=False,
    )

```

```

unit_price: Mapped[Decimal] = mapped_column(
    Numeric(14, 2),
    nullable=False,
)

subtotal: Mapped[Decimal] = mapped_column(
    Numeric(14, 2),
    nullable=False,
)

product = relationship("Product")

```

36. Pydantic Schemas

User

```

from uuid import UUID
from pydantic import BaseModel, ConfigDict

class UserResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    phone: str
    email: str | None
    status: str
    phone_verified: bool

```

37. User Profile Schema

```

class UserProfileCreate(BaseModel):
    first_name: str
    last_name: str
    username: str | None = None
    bio: str | None = None

class UserProfileResponse(UserProfileCreate):

```

```
model_config = ConfigDict(from_attributes=True)

id: UUID
user_id: UUID
avatar_url: str | None
```

38. Business Schemas

```
class BusinessCreate(BaseModel):
    name: str
    category_id: UUID | None = None
    description: str | None = None
    phone: str | None = None
    email: str | None = None

class BusinessResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    name: str
    slug: str
    description: str | None
    phone: str | None
    email: str | None
    logo_url: str | None
    verification_status: str
    rating: Decimal
    review_count: int
```

39. Product Schemas

```
class ProductCreate(BaseModel):
    name: str
    description: str | None = None
    price: Decimal
    currency: str = "UGX"
    stock_quantity: int | None = None
```

```
class ProductResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    business_id: UUID
    name: str
    slug: str
    description: str | None
    price: Decimal
    currency: str
    stock_quantity: int | None
    status: str
```

40. Order Schemas

Creating an order:

```
class OrderItemCreate(BaseModel):
    product_id: UUID
    quantity: int

class OrderCreate(BaseModel):
    business_id: UUID
    items: list[OrderItemCreate]
    delivery_address: str | None = None
    customer_note: str | None = None
```

Response:

```
class OrderItemResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    product_id: UUID
    quantity: int
    unit_price: Decimal
    subtotal: Decimal

class OrderResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
```

```
id: UUID
customer_id: UUID
business_id: UUID
status: str

subtotal: Decimal
delivery_fee: Decimal
total: Decimal
currency: str

delivery_address: str | None
customer_note: str | None

items: list[OrderItemResponse]
```

41. Payment Schemas

Request:

```
class PaymentCreate(BaseModel):
    order_id: UUID
    method: Literal[
        "MTN_MOMO",
        "AIRTEL_MONEY",
        "CARD",
    ]
    phone: str
```

Response:

```
class PaymentResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    order_id: UUID
    amount: Decimal
    currency: str
    method: str
    status: str
    provider_reference: str | None
```

42. Chat Schemas

```
class ConversationCreate(BaseModel):
    business_id: UUID | None = None
    member_ids: list[UUID]

class MessageCreate(BaseModel):
    message: str

class MessageResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    conversation_id: UUID
    sender_id: UUID
    message: str | None
    attachment_url: str | None
    is_read: bool
    created_at: datetime
```

43. Review Schemas

```
class ReviewCreate(BaseModel):
    business_id: UUID
    order_id: UUID | None = None
    rating: int
    comment: str | None = None

class ReviewResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)

    id: UUID
    user_id: UUID
    business_id: UUID
    rating: int
    comment: str | None
    created_at: datetime
```

44. API Contract

Base URL:

```
/api/v1
```

Authentication

```
POST /auth/request-otp
POST /auth/verify-otp
POST /auth/refresh
POST /auth/logout
GET  /auth/me
```

Request:

```
{
  "phone": "+256700000000"
}
```

OTP verification:

```
{
  "phone": "+256700000000",
  "otp": "123456"
}
```

Response:

```
{
  "access_token": "...",
  "refresh_token": "...",
  "token_type": "bearer"
}
```

45. User API

```
GET    /users/me
PATCH /users/me
GET    /users/{user_id}
```

Example:

```
GET /api/v1/users/me
Authorization: Bearer <token>
```

Response:

```
{
  "id": "uuid",
  "phone": "+256700000000",
  "email": null,
  "status": "ACTIVE",
  "profile": {
    "first_name": "Aron",
    "last_name": "Kolyanga",
    "username": "aron"
  }
}
```

46. Business API

```
POST    /businesses
GET     /businesses
GET     /businesses/{business_id}
PATCH  /businesses/{business_id}
DELETE  /businesses/{business_id}

POST    /businesses/{business_id}/members
GET     /businesses/{business_id}/members

POST    /businesses/{business_id}/verify
```

Example:

```
GET /api/v1/businesses?category=cakes&city=Kampala
```

Response:

```
{
  "items": [
    {
      "id": "uuid",
      "name": "FynCakes",
      "rating": 4.8,
      "review_count": 125,
      "verification_status": "VERIFIED"
    }
  ],
  "page": 1,
  "page_size": 20,
  "total": 1
}
```

47. Product API

```
POST   /businesses/{business_id}/products
GET    /businesses/{business_id}/products

GET    /products/{product_id}

PATCH /products/{product_id}

DELETE /products/{product_id}

POST   /products/{product_id}/images
```

48. Search API

```
GET /search
```

Example:

```
GET /api/v1/search?q=birthday+cake&city=Kampala
```

Possible response:

```
{
  "query": "birthday cake",
  "results": [
    {
      "type": "business",
      "id": "uuid",
      "name": "FynCakes",
      "distance_km": 2.1,
      "rating": 4.8
    },
    {
      "type": "product",
      "id": "uuid",
      "name": "Birthday Cake",
      "price": 120000,
      "currency": "UGX"
    }
  ]
}
```

Later this endpoint can become the foundation for Wamu AI.

49. Order API

```
POST /orders
GET /orders
GET /orders/{order_id}

PATCH /orders/{order_id}/cancel

POST /orders/{order_id}/confirm
POST /orders/{order_id}/ready
POST /orders/{order_id}/deliver
POST /orders/{order_id}/complete
```

Important:

The client should **not** be allowed to arbitrarily set:

```
{  
  "status": "DELIVERED"  
}
```

The backend must control state transitions.

50. Payment API

```
POST /payments  
GET /payments/{payment_id}  
POST /payments/{payment_id}/refund  
  
POST /payments/webhooks/mtn  
POST /payments/webhooks/airtel
```

Payment flow:

```
Customer  
  |  
  v  
Create Order  
  |  
  v  
Create Payment  
  |  
  v  
Payment Provider  
  |  
  v  
Customer confirms payment  
  |  
  v  
Provider Webhook  
  |  
  v  
Wamu verifies webhook  
  |  
  v  
Payment = SUCCESS  
  |
```

```
    v
    Order = CONFIRMED
```

Do not mark a payment successful merely because the mobile app says payment succeeded.

51. Chat API

REST:

```
POST /conversations
GET  /conversations
GET  /conversations/{conversation_id}
GET  /conversations/{conversation_id}/messages
POST /conversations/{conversation_id}/messages
```

WebSocket:

```
/ws/chat/{conversation_id}
```

Architecture:

```
Flutter
 |
WebSocket
 |
FastAPI
 |
Redis
 |
Chat Service
 |
PostgreSQL
```

52. Reviews API

```
POST  /reviews
GET   /businesses/{business_id}/reviews
```

```
PATCH /reviews/{review_id}
DELETE /reviews/{review_id}
```

53. Notifications API

```
GET /notifications
POST /notifications/{notification_id}/read
POST /notifications/read-all
```

54. Pagination

All list APIs should use:

```
page
page_size
```

Example:

```
GET /businesses?page=1&page_size=20
```

Response:

```
{
  "items": [],
  "page": 1,
  "page_size": 20,
  "total": 250,
  "total_pages": 13
}
```

Later, high-volume endpoints should move to cursor pagination.

55. API Error Format

Every API should return a consistent structure.

Example:

```
{
  "error": {
    "code": "BUSINESS_NOT_FOUND",
    "message": "Business was not found."
  }
}
```

Examples:

```
AUTH_INVALID_OTP
AUTH_TOKEN_EXPIRED
BUSINESS_NOT_FOUND
BUSINESS_NOT_VERIFIED
PRODUCT_NOT_FOUND
PRODUCT_OUT_OF_STOCK
ORDER_NOT_FOUND
ORDER_INVALID_STATUS
PAYMENT_FAILED
PAYMENT_ALREADY_COMPLETED
PERMISSION_DENIED
VALIDATION_ERROR
```

56. Order State Machine

Do not allow random transitions.

```
PENDING
  |
  v
CONFIRMED
  |
  v
PROCESSING
  |
  v
READY
  |
  v
OUT_FOR_DELIVERY
  |
```

v
DELIVERED

Cancellation:

PENDING ———> CANCELLED
CONFIRMED ———> CANCELLED

Refund:

DELIVERED
|
v
REFUNDED

The service layer should enforce these transitions.

57. Payment Abstraction

Do not hard-code MTN logic throughout the application.

Create:

```
from abc import ABC, abstractmethod

class PaymentProvider(ABC):

    @abstractmethod
    async def initiate_payment(
        self,
        amount: Decimal,
        phone: str,
        reference: str,
    ):
        pass

    @abstractmethod
    async def check_status(
        self,
        reference: str,
    ):
        pass
```

```

        pass

    @abstractmethod
    async def refund(
        self,
        reference: str,
        amount: Decimal,
    ):
        pass

```

Then:

```

PaymentProvider
|
+----- MTNProvider
|
+----- AirtelProvider
|
+----- CardProvider

```

This makes it possible to change payment providers without rewriting the order system.

58. Important Database Indexes

The following should be indexed:

```

users.phone
users.email

businesses.owner_id
businesses.category_id
businesses.status
businesses.verification_status

products.business_id
products.status
products.name

orders.customer_id
orders.business_id
orders.status
orders.created_at

```

```
payments.order_id
payments.user_id
payments.status

messages.conversation_id
messages.created_at

reviews.business_id

notifications.user_id
notifications.is_read
```

Do not blindly index every column.

Indexes improve reads but increase write/storage cost.

59. Database Transactions

Creating an order should be one transaction.

```
BEGIN

Validate business

Validate products

Read current product prices

Calculate subtotal

Calculate delivery fee

Calculate total

Create order

Create order_items

COMMIT
```

If anything fails:

ROLLBACK

Never create an order and its items in separate uncontrolled transactions.

60. Important Security Rules

Never trust Flutter

The mobile app can send:

```
{  
  "price": 1000  
}
```

but the backend must ignore the supplied price.

Instead:

```
Flutter  
  |  
  | product_id + quantity  
  v  
FastAPI  
  |  
  v  
PostgreSQL  
  |  
  | actual product price  
  v  
Calculate total
```

The server owns:

```
price  
subtotal  
delivery_fee  
total  
payment_status  
order_status
```

```
permissions
verification
```

61. API Permission Model

Example:

```
USER
├─ View businesses
├─ Search
├─ Order
├─ Pay
├─ Chat
└─ Review

BUSINESS OWNER
├─ Everything USER can do
├─ Create products
├─ Edit products
├─ View orders
├─ Update orders
└─ Manage staff

ADMIN
├─ Verify businesses
├─ Suspend accounts
├─ Review reports
├─ View platform analytics
└─ Manage categories
```

Permissions must be checked in FastAPI dependencies/services.

62. FastAPI Dependency Example

```
async def get_current_user(
    token: str = Depends(oauth2_scheme),
) -> User:
    # validate JWT
    # load user
```

```
# reject inactive/suspended user
...
```

Business authorization:

```
async def require_business_owner(
    business_id: UUID,
    current_user: User = Depends(get_current_user),
):
    # verify user owns business
    ...
```

This prevents a user from modifying another person's business simply by changing the UUID.

63. Recommended MVP Database Scope

Although the complete architecture contains many tables, the first production MVP should prioritize:

```
users
user_profiles
sessions

categories

businesses
business_members
locations

products
product_images

orders
order_items
payments

conversations
conversation_members
messages

reviews
notifications
```

```
audit_logs
```

Do not build every future Wamu feature into the first release.

64. First Database Seed Data

Create initial categories:

```
Restaurants
Cakes & Bakery
Groceries
Fashion
Beauty
Electronics
Phones & Accessories
Home Services
Construction
Automotive
Education
Health
Events
Entertainment
Professional Services
```

Then focus the initial marketplace on Kampala.

65. Recommended Build Order

Build the backend in this exact sequence:

```
PHASE 1
Database
  ↓
SQLAlchemy models
  ↓
Alembic migrations
  ↓
Seed data

PHASE 2
```

Authentication



OTP



JWT



User profiles

PHASE 3

Businesses



Categories



Business profiles



Locations



Verification

PHASE 4

Marketplace



Products



Images



Search

PHASE 5

Orders



Cart



Checkout



Order state machine

PHASE 6

Payments



Payment abstraction



MTN



Airtel



Webhooks



Idempotency

PHASE 7

Communication

↓

Conversations

↓

Messages

↓

WebSockets

↓

Notifications

PHASE 8

Trust

↓

Reviews

↓

Reports

↓

Audit logs

PHASE 9

AI

↓

Natural-language search

↓

Recommendations

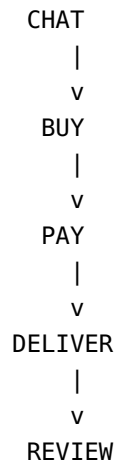
↓

AI shopping assistant

66. The Most Important Wamu Flow

The entire initial platform should eventually make this flow extremely easy:

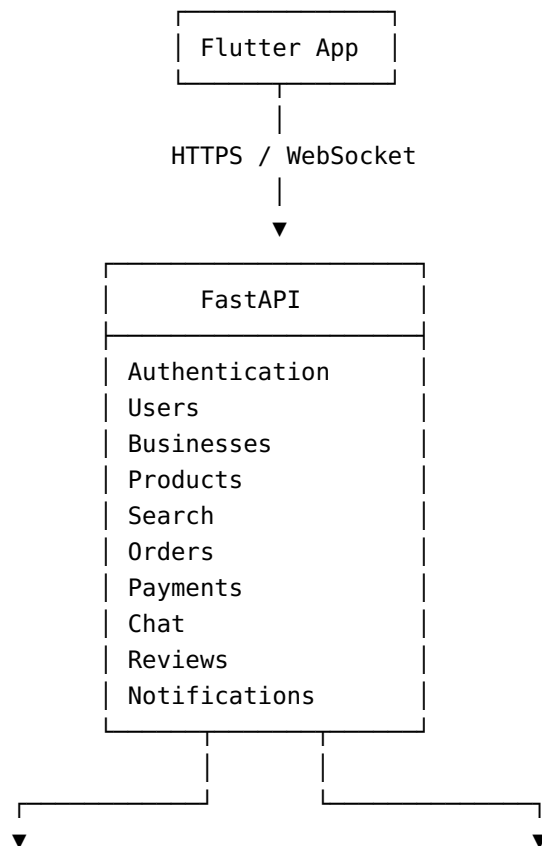
```
WAMU
|
v
SEARCH
|
v
DISCOVER
|
v
```

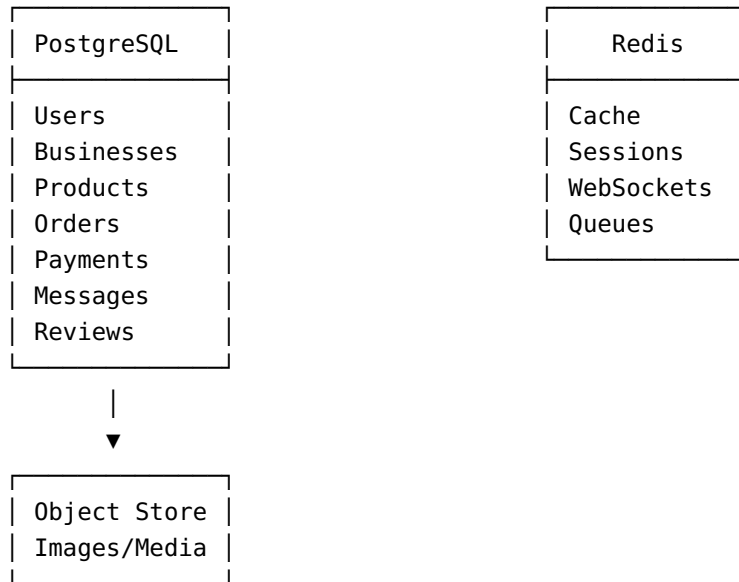


That is the first **Wamu ecosystem loop**.

The user should not have to jump between five different applications to accomplish this.

67. Final Backend Architecture





External services:

FastAPI

- |
- +---- MTN payment
- +---- Airtel payment
- +---- Push notifications
- +---- Maps/location
- +---- AI provider

68. Part 2 Deliverables

At this point the Wamu backend has:

- ✓ PostgreSQL architecture
- ✓ Database tables
- ✓ Primary keys
- ✓ Foreign keys
- ✓ Constraints
- ✓ Indexes
- ✓ Relationships
- ✓ ER diagram
- ✓ SQL schema
- ✓ SQLAlchemy models
- ✓ Pydantic schemas
- ✓ Authentication API

- ✓ Business API
- ✓ Product API
- ✓ Search API
- ✓ Order API
- ✓ Payment API
- ✓ Chat API
- ✓ Review API
- ✓ Notification API
- ✓ Error structure
- ✓ Order state machine
- ✓ Payment abstraction
- ✓ Permission model
- ✓ Security rules
- ✓ Development sequence

The next engineering stage should be **Part 3: actually creating the Wamu FastAPI project**, including `pyproject.toml`, PostgreSQL + Docker Compose, SQLAlchemy async setup, Alembic migrations, authentication/OTP, JWT, the first models, repositories, services, routers, tests, and a working `/health` endpoint.

That will turn this from a design into a runnable backend.